HDOC- 13 – 1
NAME : DARSHIL RAJ
DATE : 15/10/26 SAP ID: 590040196

Program to Print Numbers from 1 to n

This document provides a clean, well-commented implementation for printing the numbers from 1 to n in C. The program takes the value of n as input and uses a for loop to print every number from 1 up to n.

1. C Implementation

Step 1: Understand the Logic

- A for loop repeats a block of code while its condition is true. It has three parts: initialization, condition and update.
- Initialization: i = 1, so the counting starts from 1.
- Condition: i <= n, so the loop keeps running until i becomes greater than n.
- Update: i++, so i increases by 1 after every pass through the loop.
- Inside the loop, the current value of i is printed, so the numbers 1, 2, 3, ... n are printed one after another.
- If n is less than 1, there is nothing to print, so the input is rejected with an error message.

Step 2: Define Variables and Data Types

- Use int variables for the limit (n) and the loop counter (i).

Step 3: Take User Input

- Display a prompt using printf("Enter the value of n: "), then read the value using scanf("%d", &n).

Step 4: Validate the Input

- If n < 1, print an error message and do not run the loop.

Step 5: Print the Numbers Using a for Loop

1. Start with i = 1.
2. While i <= n, print i followed by a space.
3. Increase i by 1 and check the condition again.
4. When i becomes greater than n, the loop ends. Print a new line.

Complete C Code

```c
/*
 * Q21 (User Inputs, Operations & Output)
 * Write a program to print numbers from 1 to n.
 */
#include <stdio.h>

int main() {
    int n, i;

    // Step 1: Read the value of n
    printf("Enter the value of n: ");
    scanf("%d", &n);

    // Step 2: Validate n
    if (n < 1) {
        printf("Invalid input! n must be at least 1.\n");
    } else {
        // Step 3: Print the numbers from 1 to n using a for loop
        printf("Numbers from 1 to %d:\n", n);
        for (i = 1; i <= n; i++) {
            printf("%d ", i);
        }
        printf("\n");
    }

    return 0;
}
```

Program Output (Sample Runs)

Run 1:

```
$ gcc hdoc13_1.c -o hdoc13_1
$ ./hdoc13_1
Enter the value of n: 5
Numbers from 1 to 5:
1 2 3 4 5
```

Run 2:

```
$ ./hdoc13_1
Enter the value of n: 10
Numbers from 1 to 10:
1 2 3 4 5 6 7 8 9 10
```

Run 3:

```
$ ./hdoc13_1
Enter the value of n: 1
Numbers from 1 to 1:
1
```

Run 4:

```
$ ./hdoc13_1
Enter the value of n: 0
Invalid input! n must be at least 1.
```

Dry Run with Test Cases

Sample Test Case 1:

- Input: n = 5
- Calculation:
  - 5 < 1 is FALSE, so the input is valid and the loop runs
  - Start: i = 1
  - Pass 1: 1 <= 5 is TRUE, print 1, then i becomes 2
  - Pass 2: 2 <= 5 is TRUE, print 2, then i becomes 3
  - Pass 3: 3 <= 5 is TRUE, print 3, then i becomes 4
  - Pass 4: 4 <= 5 is TRUE, print 4, then i becomes 5
  - Pass 5: 5 <= 5 is TRUE, print 5, then i becomes 6
  - Check: 6 <= 5 is FALSE, so the loop ends
- Output: 1 2 3 4 5

Sample Test Case 2:

- Input: n = 1
- Calculation:
  - 1 < 1 is FALSE, so the input is valid and the loop runs
  - Start: i = 1
  - Pass 1: 1 <= 1 is TRUE, print 1, then i becomes 2
  - Check: 2 <= 1 is FALSE, so the loop ends after one pass
- Output: 1

Sample Test Case 3:

- Input: n = 0
- Calculation:
  - 0 < 1 is TRUE, so the input is invalid
  - The error message is printed and the loop does not run
- Output: Invalid input! n must be at least 1.